

Lesson 10.1 — Шпаргалка з демо (Sanctum auth-флой)

Це **довідковий зріз коду**, який ми написали вживу під час Google Meet на демо-проєкті **practice-laravel**. Файл потрібен для того, щоб ви могли:

- швидко скопіювати фрагмент, якщо щось не встигли занотувати;
- порівняти свій варіант із тим, що було на екрані;
- спокійно йти у [lesson_10_1_practice_students.md](#) і [homework_10_1.md](#) на власному CRM.

Важливо. Демо ми робили у **workflow/practice-laravel/** (домен Payment/Account, базовий URL **<http://localhost>**). У вашій домашці ви робите те саме у власному CRM (домен Client/Invoice/Transfer/Service, базовий URL **<http://localhost:8000>**, інші назви FormRequest'ів і додаткові поля **role/client_id**). Тому це **не «копія-паст у свій проєкт»**, а орієнтир: «ось так виглядає той самий патерн на простішому проєкті». Точні вимоги до вашого CRM — у [lesson_10_1.md](#) і практикумі.

0) Що вже є у демо-проєкті до старту

- Sanctum встановлений (**composer.json**, **config/sanctum.php**, **bootstrap/app.php**).
- Таблиця **personal_access_tokens** мігрована.
- Стандартний **GET /api/user** під **auth:sanctum** від Laravel.
- **User** модель — стандартний **Authenticatable**, **password** хешується через cast **'hashed'**.

1) Блок А — **tinker**: видати і подивитись токен

Запускаємо REPL:

```
docker compose exec -T php php artisan tinker
```

Усередині:

```
$user = \App\Models\User::factory()->create([
    'email' => 'demo@example.com',
    'password' => \Illuminate\Support\Facades\Hash::make('secret123'),
]);

$token = $user->createToken('demo-meet')->plainTextToken;
echo $token;

\App\Models\User::where('email', 'demo@example.com')->first()->tokens()->count();
exit
```

Ключове, що ми показували:

- `plainTextToken` повертається **один раз** — у БД у `personal_access_tokens.token` зберігається лише SHA-256 хеш.
- Втратили токен — видаємо новий і відкликаємо старий через `tokens()->delete()`.

Чистка демо-користувача перед наступним блоком:

```
docker compose exec -T php php artisan tinker --  
execute="\App\Models\User::where('email', 'demo@example.com')->delete();" 
```

2) Блок В — `AuthController::register/login`

Генеруємо каркас:

```
docker compose exec -T php php artisan make:controller  
Api/V1/AuthController
```

`app/Http/Controllers/Api/V1/AuthController.php` після наповнення:

```
<?php  
  
declare(strict_types=1);  
  
namespace App\Http\Controllers\Api\V1;  
  
use App\Models\User;  
use Illuminate\Http\JsonResponse;  
use Illuminate\Http\Request;  
use Illuminate\Routing\Controller;  
use Illuminate\Support\Facades\Hash;  
use Illuminate\Validation\ValidationException;  
  
class AuthController extends Controller  
{  
    public function register(Request $request): JsonResponse  
    {  
        $data = $request->validate([  
            'name' => ['required', 'string', 'max:120'],  
            'email' => ['required', 'email', 'unique:users,email'],  
            'password' => ['required', 'string', 'min:8'],  
        ]);  
  
        $user = User::create([  
            'name' => $data['name'],  
            'email' => $data['email'],  
            'password' => Hash::make($data['password']),  
        ]);  
  
        $token = $user->createToken('api')->plainTextToken;
```

```

        return response()->json([
            'data' => [
                'user' => ['id' => $user->id, 'email' => $user->email],
                'token' => $token,
            ],
        ], 201);
    }

    public function login(Request $request): JsonResponse
    {
        $data = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required', 'string'],
        ]);

        $user = User::where('email', $data['email'])->first();

        if ($user === null || ! Hash::check($data['password'], $user->password)) {
            throw ValidationException::withMessages([
                'email' => 'Невірні дані для входу.',
            ]);
        }

        $token = $user->createToken('api')->plainTextToken;

        return response()->json([
            'data' => [
                'user' => ['id' => $user->id, 'email' => $user->email],
                'token' => $token,
            ],
        ], 200);
    }
}

```

Що варто запам'ятати з цього шматка:

1. `register` повертає `201 Created`, `login` — `200 OK`.
2. На неправильний пароль кидаємо `ValidationException` — Laravel віддасть `422` із полем `email`.
3. Текст помилки **єдиний** для «email не знайдено» і «пароль не підійшов» — захист від enumerate-атаки.
4. `createToken('api')` — ім'я `api` потрібне лише для адмінки/Telescope, на безпеку не впливає.

Маршрути у `routes/api.php`

```

use App\Http\Controllers\Api\V1\AuthController;

Route::prefix('v1')->group(function () {

```

```
// Публічні auth-маршрути – без auth:sanctum
Route::middleware('throttle:auth')->group(function () {
    Route::post('auth/register', [AuthController::class, 'register'])->
>name('auth.register');
    Route::post('auth/login', [AuthController::class, 'login'])->
>name('auth.login');
});

// ... решта існуючих v1-маршрутів залишається ...
});
```

Іменованний rate-limiter auth

У `app/Providers/AppServiceProvider.php`, метод `boot()`:

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;

RateLimiter::for('auth', function (Request $request): Limit {
    return Limit::perMinute(10)->by((string) $request->ip());
});
```

Чому саме **by IP**: у гостя ще немає `user_id`. 10 запитів/хвилину достатньо для нормального флоу, але «вбиває» брутфорс на 11-му запиті.

Live-перевірка register/login

```
# 1) Реєстрація – отримуємо токен
curl -i -X POST http://localhost/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"name":"Demo","email":"demo@example.com","password":"secret123"}'

# 2) Логін – отримуємо новий токен (старий продовжує діяти)
curl -i -X POST http://localhost/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"demo@example.com","password":"secret123"}'

# 3) Логін з неправильним паролем – 422 + загальна помилка
curl -i -X POST http://localhost/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"demo@example.com","password":"wrong"}'
```

3) Блок C — auth:sanctum + /me + logout

Допишуємо у `AuthController`:

```
public function me(Request $request): JsonResponse
{
    $user = $request->user();

    return response()->json([
        'data' => [
            'id' => $user->id,
            'email' => $user->email,
            'name' => $user->name,
        ],
    ]);
}

public function logout(Request $request): JsonResponse
{
    $request->user()->currentAccessToken()->delete();

    return response()->json(null, 204);
}
```

Маршрути — поруч із реєстрацією, але **не** в групі `throttle:auth`:

```
Route::middleware('auth:sanctum')->group(function () {
    Route::get('auth/me', [AuthController::class, 'me'])->name('auth.me');
    Route::post('auth/logout', [AuthController::class, 'logout'])->
    >name('auth.logout');
});
```

Ключове:

- `currentAccessToken()->delete()` зносить **тільки поточний** токен — інші пристрої залишаються залогінені.
- «Вийти з усіх пристроїв» — це окрема дія `tokens()->delete()`, на окремому маршруті.

Live-перевірка захищених маршрутів

Замість `<TOKEN>` — токен із Блоку В.

```
# 1) /me з валідним токеном — 200 + дані юзера
curl -i http://localhost/api/v1/auth/me \
  -H "Authorization: Bearer <TOKEN>"

# 2) /me без токена — 401
curl -i http://localhost/api/v1/auth/me

# 3) /me з зіпсованим токеном — 401
curl -i http://localhost/api/v1/auth/me \
  -H "Authorization: Bearer 999|definitely_not_valid"
```

```
# 4) Logout — 204, токен видаляється
curl -i -X POST http://localhost/api/v1/auth/logout \
  -H "Authorization: Bearer <TOKEN>"

# 5) Те саме /me з тим самим токеном — тепер уже 401
curl -i http://localhost/api/v1/auth/me \
  -H "Authorization: Bearer <TOKEN>"
```

4) Блок D — як це ламається

Демо ліміту на **login**

```
for i in {1..15}; do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST
  http://localhost/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email":"demo@example.com","password":"wrong"}'
done
```

Очікуємо: перші ~10 запитів — 422, далі — 429 Too Many Requests із заголовком **Retry-After**.

Expiration токенів

у **config/sanctum.php**:

```
'expiration' => null,
```

- **null** — токен живе вічно (зручно для демо).
- **60** — токен невалідний через 60 хвилин (для production-фінтеху обов'язково).
- У ДЗ ставимо **SANCTUM_TOKEN_EXPIRATION_MINUTES=240** у **.env** і перевіряємо, що **config/sanctum.php** читає це значення.

5) Фінальний чек-лист auth у production

- зберігаємо тільки **хеш** токена/пароля (Sanctum + **Hash::make**)
- публічні **register/login** — за **throttle** з лімітом за IP
- усі стейтові ендпоінти — за **auth:sanctum**
- **logout** видаляє **поточний** токен; «вийти з усіх» — окрема дія
- однакове повідомлення про помилку для невірного email/пароля
- токен ніколи не логуємо і не кладемо в URL
- expiration + revoke стратегія прописані у **config/sanctum.php**

6) Як перенести це у власний CRM (для домашки)

Демо було спрощене — у CRM додатково потрібно:

- розширити `User`: трейт `HasApiTokens`, поля `role/client_id` у `$fillable`, relationship `client()` (Вправа 1 практикуму).
- міграція `add_role_and_client_to_users_table` (Вправа 2).
- FormRequest-класи `RegisterRequest` і `LoginRequest` замість inline-валідації через `$request->validate(...)` (Вправа 3).
- метод `me` має повертати `id, name, email, role` (а не лише `id/email/name` як у демо).
- маршрути ділимо на **чотири** блоки: публічний `throttle:5,1` для `auth`, публічний `throttle:120,1` для `services`, закритий `auth:sanctum + throttle:60,1` для ресурсів, закритий `auth:sanctum + throttle:5,1` для `reports/*` (Вправа 4).
- адаптуємо `TransferApiTest` під `Sanctum::actingAs(...)` (Вправа 6).

Точні приклади коду під CRM-проект — у [lesson_10_1.md](#) (Лістинги 10.1.1–10.1.7). Покрокові завдання — у [lesson_10_1_practice_students.md](#). Обсяг домашки — у [homework_10_1.md](#).